Available online at www.sciencedirect.com

Procedia Computer Science 00 (2026) 1–11

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

Oasis: A Mutation Testing Framework for Evaluating the Semantic Quality of Infrastructure-as-Code Test Suites

Tushar Teotia[a], Shiven Goomer[a], Yash Vardhan Rawat[a], Minakshi Tomer[a,2,*], Manoj Kumar Malik[a,1]

[a]*Department of Information Technology, Maharaja Surajmal Institute of Technology, Janakpuri, New Delhi 110058, India*

Abstract

Passing Infrastructure-as-Code tests does not establish that assertions detect configuration changes. Oasis applies a frozen 16-operator catalog to native Terraform and OpenTofu suites, separating baseline, validity, assertion, runtime-error and restoration evidence. From a technically screened convenience sample, the primary dataset contains 62 archived repository attempts and 47 scored repositories: 1288 scored mutant cases comprise 115 assertion kills, 76 runtime-error kills and 1097 survivors. Known selected-context no-ops and unresolved bindings are excluded conservatively; general semantic equivalence remains unsolved. The assertion-kill fraction is 8.9%; combined rejection is 14.8%. We retain 245 excluded mutant records and distinguish full from partial scope coverage, blocked baselines and unrun repositories under a disclosed deadline cutoff. Generic Boolean edits contribute 89.1% of scored cases, limiting security-specific interpretations. A separate frozen 123-case study raises assertion detections from 39 to 73 with no losses. Historical catalog 2 results and the scanner comparison retain separate denominators. These exploratory observations concern sensitivity under recorded fixtures, not representative fault prevalence, dynamic resource coverage or deployed-cloud security.

© 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).
Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: Infrastructure-as-Code, Terraform, OpenTofu, Mutation Testing, Test Oracles, Reproducibility

1. Introduction

Infrastructure-as-Code (IaC) places infrastructure decisions under version control, but a valid configuration can still express an undesirable access, encryption or capacity setting. Research on defect-related properties in Puppet [1], security smells [2] and policy adoption in Terraform [3] motivates configuration testing without establishing how sensitive a particular project's assertions are to changes in its intended configuration.

*Corresponding author. Email: tomer.minakshi@gmail.com
[1]Assistant Professor. Email: manojmalik@msit.in. ORCID: 0000-0001-9477-5592.
[2]Assistant Professor. ORCID: 0000-0001-5536-3675.

Terraform and OpenTofu support native tests over plans and applies [4, 5]; provider mocks supply controlled values instead of remote operations [6]. A passing suite can therefore establish anything from configuration executability to a specific invariant. Mutation testing measures sensitivity by changing a program and observing its tests [7, 8]. The principle and literal replacements are established techniques. Oasis contributes native-suite integration, selected-scope and caller/default handling, separate execution gates, bounded configuration operators and an auditable empirical account. We do not claim that adding operators alone establishes a new testing principle.

We address four questions: **RQ1**, which repository scopes produce usable outcomes and which are blocked? **RQ2**, how do the executed operator families and retained/added subsets behave? **RQ3**, can test strengthening change detections on identical frozen cases? **RQ4**, what costs are observed, and how do historical native outcomes relate to policy-scanner observations? The primary expansion, historical studies and matched strengthening experiment are reported separately.

2. Background and related work

IaC research relates source properties to defects [1], identifies security smells [2], and analyzes input-handling and configuration bugs across Ansible, Puppet and Chef [9]. Verdet et al. study security-policy adoption in public Terraform projects [3]. These findings motivate bounded configuration fault models; neither cross-ecosystem taxonomies nor policy categories validate the prevalence or realism of our exact mutations. GLITCH detects static IaC smells [10, 11], a complementary purpose to observing an existing suite's response to an edit.

Table 1. Different testing targets in related work. This is a capability/estimand comparison, not a controlled speed or accuracy ranking.

Approach	Object and oracle	Relationship to Oasis
ProTI [12]	Pulumi TypeScript programs; generated configurations, mocks and pluggable property oracles	Generates test configurations rather than evaluating an existing native HCL suite against configuration-changing edits.
EMIaC [13]	IaC engines; consistency under semantics-preserving program transformations	Targets engine inconsistencies; Oasis instead observes suite rejection of intended configuration perturbations.
Oasis	Native Terraform/OpenTofu suites; existing assertions and runtime outcomes	Combines literal edits, selected-scope and effective-input gates, outcome separation and recorded restoration.

ProTI generates and checks configurations for Pulumi TypeScript programs using mocks and pluggable oracles [12]. EMIaC uses semantics-preserving transformations to test IaC engines [13]. Oasis instead changes intended configuration values and invokes existing native suites. These distinctions replace a broad claim that IaC mutation testing is wholly unexplored.

Mutation surveys discuss operator selection, execution cost and semantic equivalence [7, 8, 14]. Application-code research investigates coupling between mutant and real-fault detection [15], limitations of coverage as an effectiveness proxy [16], industrial testing practice [17, 18], and practitioner adoption [19]. Such work motivates the technique but supplies no universal adequate-score threshold for this corpus.

Table 2. Different experimental purposes preclude a common mutation-score ranking.

Study	Evidence focus	Comparison limit
Application mutation [15]	Mutant–real-fault relationship	No comparable real Terraform fault oracle here.
Industrial mutation [17, 18]	Testing practice and selective execution at scale	Different operators, sampling and filtering.
Oasis	1288 primary cases; separate 123 matched cases	Native mock/fixture sensitivity; clustered observations.

3. Oasis design

3.1. Execution pipeline and outcome gates

Oasis is implemented in Python. It identifies supported literal spans in tracked local HCL, records the file, byte offset, original and replacement values, and source hashes, and applies one edit at a time without reserializing surrounding configuration. A scope comprises a repository, module directory and selected test directory. Downloaded registry modules can execute as dependencies but lie outside the local-source mutation pool. The same edit exercised through two scopes is two mutant cases, not two independent real defects.

The pipeline initializes dependencies, runs unmutated native tests and standalone validation, checks the candidate's effective input context, validates each mutation, invokes native tests, classifies the outcome and restores source bytes. A baseline failure is never a mutant kill. Timeouts, invalid candidates, unresolved contexts and known no-ops remain explicit exclusions. OS sandboxing restricts native writes and private-file access; native evaluation is offline except local provider IPC. Provider downloads occur only during serialized initialization. No cloud credentials or live cloud provisioning are used.

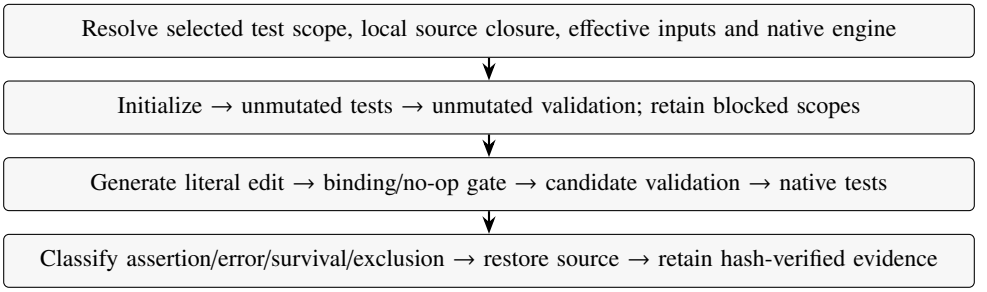


Fig. 1. Native mutation pipeline. Static potential reachability is not dynamic coverage.

3.2. Frozen catalog and fault-model boundaries

The primary study freezes framework `0.3.0-research.1`, catalog 3, 16 identifiers and 22 source-file hashes before native expansion. Twelve retained identifiers and four added identifiers are described in Table 3. Identical offset/replacement/deletion edits are counted once per scope, with the earlier specialized operator retaining attribution. The added generic Boolean operator targets supported resource/local literals, module input arguments (M), and scalar variable defaults (V). This extends bounded module-input coverage, not arbitrary module-interface semantics.

Table 3. Frozen catalog 3. Transformations are bounded fault hypotheses, not a validated distribution of real bugs. R: resource, V: variable default, L: local, M: module argument.

Identifier(s)	Transformation	Surface
MUT-SEC-001	S3 private ACL to public-read	R
MUT-SEC-002	Supported storage encryption true to false	R
MUT-SEC-003	Restrictive literal CIDR to 0.0.0.0/0	R
MUT-SEC-004a–d	Disable one of four S3 public-access-block flags	R
MUT-RES-001	Supported instance-size literal to a larger class	R,V
MUT-RES-002	Supported count/desired/minimum of at least two to one	R,V
MUT-RES-003	Positive replica literal to zero	R,V
MUT-TAG-001	Environment production to development	R,V,L
MUT-TAG-002	Delete a literal team/owner/cost_center tag	R,L
MUT-CFG-001 (added)	Boolean literal inversion, including supported caller/default sites	R,V,L,M
MUT-RES-004 (added)	Curated positive capacity/count literal to zero	R,V
MUT-SEC-005 (added)	Scoped EC2 IMDS tokens required to optional	R
MUT-SEC-006 (added)	Scoped EFS/Redshift encryption true to false	R

Access/encryption transformations relate conceptually to security-policy categories [3]; capacity and input perturbations relate to broader IaC configuration-fault research [9]. Generic Boolean inversion is a configuration fault, not automatically a security fault. Metadata edits concern project-specific contracts. Literal matching, resource restrictions, unsupported expression/object forms, remote dependencies and shared module structure constrain the model. Native OpenTofu execution is supported, but no OpenTofu-specific semantic rewrite family is evaluated.

3.3. Equivalence and selected-context binding

Survival does not establish semantic equivalence. Catalog 3 conservatively resolves caller/default binding within the selected local test closure: an independently supplied effective value can shadow a mutated default; a missing argument, or explicit null where a non-nullable default applies, can leave that default relevant. Unresolved nullability, parser or caller cases are excluded with recorded reasons. Potential input influence is not proof that the changed attribute executes or that an assertion observes it.

For illustration, if a non-nullable Boolean default changes from true to false but every selected caller independently supplies true, the default edit is shadowed in those contexts. A caller that omits the argument can instead expose the edit. This is a binding example, not an additional experimental case or proof of global equivalence.

We do not remove survivors because a scanner is silent, an attribute lacks a direct reference, or a mock hides the difference. Unknown equivalent mutants remain in the denominator. Historical optional labels such as `no_coverage` describe static reference patterns only; they are not dynamic resource coverage and do not determine scoring.

4. Empirical method

4.1. Selection, cohorts and deadline cutoff

The primary plan contains 95 distinct canonical upstream identities: 37 earlier additions, 35 technically applicable reserves from 40 completed static screens, and 23 original sources retained in their historically adapted form. Discovery and manual inspection favor native tests, mock-compatible execution and local configurations with potential frozen-operator sites. Static screening stops at a first unshadowed potential site and records a lower bound; it neither predicts the final mutant count nor substitutes for native baseline gates. Admission does not use a kill-rate threshold; earlier campaign outcomes are nevertheless visible. This convenience sample includes reusable modules, platforms, examples and deliberately vulnerable fixtures; it is not representative of Terraform users.

A time-based submission cutoff closes the queue before the final LangChain GCP and n8n outcomes are known. Only those already active repositories are allowed to finish; other repositories and pending preparation/environment retries are not continued. Earlier outcomes and runtimes were visible when this decision was made, and the scheduling favors smaller observed workloads. The cutoff is therefore disclosed selection/attrition, not random sampling, a blind holdout, a claim that all 95 ran, or outcome-independent preregistration. The original plan and every disposition are retained in the supplement. At this dataset snapshot, 62 attempts are archived, 3 preparation failures are retained with retries cancelled, and 29 repositories have not started.

4.2. Source policy, engines and execution

The primary expansion makes no new changes to upstream assertions, mocks or test values. The original 23 cohort already contains historical execution adaptations and is explicitly labelled accordingly. Earlier adaptations included synthetic inputs, mock/alias wiring, artifact fixtures and diagnosed source compatibility changes; thus these sources are not pristine upstream suites. Their adaptation patches and original/final baseline ledgers remain separate historical evidence.

Primary native execution uses Terraform 1.16.7 or OpenTofu 1.12.3, two lanes, serialized initialization, a 900-second initialization limit and 600-second per-command test/validation limits. Every eligible mutant in a baseline-cleared selected scope is attempted without a mutant-count cap. Source restoration is checked after scopes. SHA-verified immutable APFS provider snapshots prevent concurrent package-cache writes from changing running provider binaries. Initialization failures, native test failures, blocked input contexts and timeouts are retained rather than repaired by weakening assertions.

Storage interruptions and the later queue-cutoff handoff preserve original attempts. A scope is reused only after exact selected command, source revision/file hashes, frozen code hashes and manifest/artifact verification; interrupted attempts remain outside active counts. The two active native workers continue across the cutoff handoff without restarting their commands. One separate GCP baseline timing probe briefly used an authorized third lane, ran no mutants and contributes no primary cases, scope completions or command-cost observations.

4.3. Metrics and denominators

Let A denote assertion kills, E runtime-error kills and S survivors among valid, baseline-cleared scored cases:

$$N = A + E + S, \qquad D_A = A/N, \qquad MS = (A + E)/N. \tag{1}$$

D_A measures observed assertion sensitivity; MS additionally includes runtime rejection. Timeout is not credited as either kind of kill. Invalid, no-op, unresolved, timeout and execution-error mutant records are separately excluded. Baseline failures and target-free module rows are scope dispositions, not generated mutant cases. Repositories with $N = 0$ receive n.a., not a zero score. Pooled case fractions, repository-level summaries and all-selected-baseline coverage answer different questions. A finished scored repository can still contain blocked scopes.

The retained 12 and added 4 subsets use the same catalog 3 engine, inputs, resolver and dependency context. This is a within-version operator partition, not a controlled catalog 2 rerun. Historical cases, the matched strengthening pool and the primary expansion never share one mutation denominator. We report canonical identity unions only as provenance, not as a combined scored sample.

4.4. Separate historical and matched studies

The original catalog 2 execution-adapted study covers 23 repositories and 336 scope dispositions:306 pass baselines,27 fail and 3 lack dependencies. It generates 299 cases, with 276 scored (47 assertion kills,2 runtime kills,227 survivors),21 unresolved,1 invalid and 1 proven context no-op. A prior 37-repository catalog 2 expansion contributes 283 scored cases in 13 repositories; exact-environment recovery adds Worldcoin's 3 scored survivors. These historical sets account for 29 scored identities, with their original manifests, engine versions and preparation policies; their mutation percentages are not pooled with catalog 3.

A separate experiment freezes 123 matched cases across 40 modules in 8 repositories. The original and strengthened arms use exactly the same edits. Source-informed strengthening adds 63 assertions to existing runs and 37 positive-control runs while preserving original runs, mocks, overrides and source. All 40 strengthened baseline/control gates pass. The development subset contains 95 cases and the transfer subset 28 cases from 2 repositories. Proposals are sealed before transfer execution, but prior pilot exposure and investigator-authored treatment preclude a pristine blind-holdout claim. All 80 native arm jobs finish, yielding 246 arm-specific executions; historical original labels reproduce. Preservation checks performed for this revision verify bytes and paired outcomes; they do not claim fresh reruns of those 80 jobs.

4.5. Policy comparison and timing protocol

The policy comparison uses the historical original 23 catalog 2 snapshots, not the new primary cohort. tfsec 1.28.14 [20] scans copied, hash-verified sources without module downloads. Unmutated/mutant module comparisons associate newly reported rule identifiers with findings overlapping the mutation line. Existing baseline findings receive no detection credit. Location association is incomplete semantic evidence and is not vulnerability ground truth; scanner results never become assertion kills.

Primary command receipts separate initialization, baseline tests and validation, and the validation and test command for each mutant. We report elapsed seconds, medians and empirical nearest-rank 95th percentiles from recorded commands, including failures and timeouts. Summed commands overlap across lanes and omit some preparation, queue wait, snapshot and framework overhead; they are not wall time. Historical command costs retain their own cohort/version. Repeated-run variance, deployment cost savings and engine performance differences are not measured.

5. Results

5.1. RQ1: completion, coverage and exclusions

Table 4 gives the primary disposition ledger. 192 archived scope attempts comprise 130 passing and 62 blocked/unavailable baselines. 47 completed repositories contribute scored mutants: 39 pass every selected baseline; 8 contribute scored cases from some scopes while others are blocked. Another 15 completed repository attempts have no scored mutants. A repository count alone therefore overstates complete execution coverage if these categories are collapsed.

Table 4. Disposition of the 95-repository plan at the reporting snapshot. Only completed archives contribute mutant outcomes.

Disposition	Count
Completed repositories with scored mutants: all selected baselines pass	39
Completed repositories with scored mutants: some baselines blocked	8
Completed repositories without scored mutants	15
Preparation failures; retries cancelled	3
In progress; excluded from mutation totals	1
Not started before the deadline cutoff	29
Total planned repository identities	95
Completed scope attempts: passing / blocked baseline	130 / 62
Generated cases: scored / excluded	1,288 / 245

Table 5. Catalog 3 outcomes by admission cohort. The sources and sampling policies differ; these are not matched groups.

Admission cohort	Completed	Scored repos	N	D_A
Historically adapted original sources	15	11	507	7.9%
First 37 additions	20	14	379	10.6%
Applicability-screened reserves	27	22	402	8.7%

There are 1533 generated mutant-case records: 1288 scored and 245 excluded. Exclusions comprise 193 known context no-ops, 39 unresolved contexts, 13 timeouts and 0 other invalid/execution-error records. Baseline-blocked scopes can have unknown potential mutant counts; the excluded-case total is not a census of every counterfactual mutation in unexecuted scopes. The repository/scope/case CSVs retain every admitted identity and unavailable outcome.

LangChain illustrates runtime and feasibility differences: its AWS scope completes 133 eligible native attempts, including 13 timeouts, after 39 shadowed candidates are excluded; its Azure initialization encounters an HTTP504 while fetching provider authentication checksums and does not reach baseline testing. The Azure retry is cancelled under the declared cutoff. The completed AWS/GCP record supplies 233 scored cases in total; the unfinished n8n repository remains excluded from this snapshot. These facts describe the attempted environment, not Azure assertion weakness or a completed three-cloud success. Other retained blockers include missing source, provider availability/startup errors, offline external-program requirements and native source/test disagreements.

5.2. RQ2: operator and family outcomes

Primary outcomes are 115 assertion kills, 76 runtime-error kills and 1097 survivors, giving D_A =8.9% and MS =14.8%. The unweighted mean repository combined fraction is 33.5% and its median is 18.2% over repositories with scored cases. These descriptive summaries do not turn small clustered samples into independent population observations.

Table 6. Operator-family outcomes in catalog 3. Families partition scored cases, not independent real defects.

Family	N	A	E	S	D_A
Configuration	1147	102	51	994	8.9%
Security	67	9	0	58	13.4%
Resource	72	3	24	45	4.2%
Compliance	2	1	1	0	50.0%

Table 7. Within-catalog 3 operator partitions use the same execution machinery. This is not a catalog 2 versus catalog 3 performance comparison.

Subset	N	A	E	S	D_A	MS
Retained 12 identifiers	77	9	2	66	11.7%	14.3%
Added 4 identifiers	1211	106	74	1031	8.8%	14.9%

Table 8. All 16 operator identifiers, including the zero-target identifier. Exclusions are retained separately; n.a. means no scored denominator.

Identifier	Gen.	N	A	E	S	Excl.	D_A
`MUT-CFG-001`	1363	1147	102	51	994	216	8.9%
`MUT-RES-001`	19	9	0	1	8	10	0.0%
`MUT-RES-002`	8	4	0	0	4	4	0.0%
`MUT-RES-003`	3	3	0	0	3	0	0.0%
`MUT-RES-004`	67	56	3	23	30	11	5.4%
`MUT-SEC-001`	1	1	0	0	1	0	0.0%
`MUT-SEC-002`	4	4	1	0	3	0	25.0%
`MUT-SEC-003`	2	2	0	0	2	0	0.0%
`MUT-SEC-004a`	14	13	3	0	10	1	23.1%
`MUT-SEC-004b`	14	13	2	0	11	1	15.4%
`MUT-SEC-004c`	14	13	1	0	12	1	7.7%
`MUT-SEC-004d`	14	13	1	0	12	1	7.7%
`MUT-SEC-005`	7	7	1	0	6	0	14.3%
`MUT-SEC-006`	1	1	0	0	1	0	0.0%
`MUT-TAG-001`	2	2	1	1	0	0	50.0%
`MUT-TAG-002`	0	0	0	0	0	0	n.a.

The generic Boolean operator contributes 1147 cases (89.1% of scored cases), so larger sample coverage mainly reflects configuration-literal sensitivity. The retained 12 subset contains 77 scored cases; the added 4 subset contains 1211. Their different outcome fractions reflect different targeted faults and sites, not a causal effect of an improved operator design or a version-to-version change in suite quality. Security and resource families are reported separately rather than interpreting every Boolean change as a security defect. Zero-target operator rows remain visible.

Table 9. Actual recorded configuration edits, not synthetic results. Case identifiers and source locations are supplied in selected-cases.json; these examples do not establish vulnerability ground truth.

ID	Repository	Recorded edit	Outcome
C1	ambicuity/terraform-aws-eks-ray	`enable_key_rotation`: `true` → `false`	Survived
C2	equinor/terraform-azurerm-key-vault	default: `true` → `false`	Assertion kill
C3	langchain-ai/terraform	special: `false` → `true`	Timeout / excluded

A survivor shows only acceptance under recorded inputs and mocks. Missing or indirect oracles, inactive conditions, mock behavior, irrelevant contracts and semantic equivalence are possible explanations. If e scored survivors were later proved equivalent, the assertion fraction would be $A/(N - e)$; no value of e is inferred from survival or scanner silence. Neither unresolved outcomes nor unknown mutants in baseline-blocked scopes are imputed as detected or undetected.

5.3. RQ3: frozen test strengthening

Table 10. Separate fixed 123-case comparison; every pair is scored.

Subset	Cases	Original A	Strengthened A	New detections
Development	95	34	56	22
Transfer (two repositories)	28	5	17	12
Total	123	39	73	34

Assertion detections rise from 39/123(31.7%) to 73/123(59.3%), a 27.6 percentage-point gain, with 34 new and 0 lost detections. Two runtime-error kills remain in each arm; survivors fall from 82 to 48. Gains concentrate in platform-tools(20), serverless-architecture-patterns(12) and platform-design(2). Thirty-two of the 34 gains concern S3 public-access-block edits; the remaining two concern replica-to-zero edits. All 12 transfer gains occur in the first of the two transfer repositories. This is direct evidence of changed sensitivity to fixed edits, not dynamic resource coverage, general transfer to real bugs or an independence-based significance result. The treatment includes test-input/positive-control expansion, not just altered assertion expressions.

5.4. RQ4: costs and historical scanner observations

Table 11. Observed native command durations (seconds), including failures and timeouts. Interrupted attempts and the auxiliary baseline probe are excluded; sums are not wall time.

Phase	Commands	Sum (s)	Median (s)	P95 (s)
Initialization	191	1,641.1	4.57	24.97
Unmutated test	165	3,102.2	13.21	35.11
Unmutated validation	131	571.1	3.06	14.17
Mutant validation	1301	8,426.9	5.57	14.17
Mutant test	1301	94,002.5	11.59	386.88

Table 11 includes native commands from archived primary attempts and excludes interrupted history and the separate probe. Cost skew and failed setup are part of feasibility. A warm-cache fast baseline is not a full mutation-run forecast: the number of effective sites, failing paths and timeouts also matters. Exhaustive execution on important modules can be expensive; this study does not establish a mandatory routine-adoption policy or validate a cost-reduction scheme.

The following scanner observations use only the historical 276-case original 23 study. They are retained to answer the review request for a named scanner on corresponding snapshots, not extrapolated to catalog 3. The repository-root scanner invocation completes with structured output for 20 of 23 snapshots; three fail to produce usable JSON. Root-level finding counts are not a measure of complete resource coverage. At the case level, 231 unmutated/mutant comparisons are usable and 45 are unavailable (16 GenAI and 29 Psoxy cases), principally because this scanner rejects newer HCL blocks such as `check` and `removed`. Native test results for those cases remain valid; scanner failures are not treated as policy nondetections. Table 12 retains all cases.

Table 12. Joint native-test/scanner outcomes on expanded cases. "New rule" means a newly reported rule identifier overlapping the mutation line; it is not vulnerability ground truth.

Scanner comparison	Assertion kill	Runtime kill	Survivor
Usable: new site rule	38	0	88
Usable: no new site rule	9	2	94
Unavailable comparison	0	0	45
Total	47	2	227

Among the 231 usable comparisons, 126 have a new site-associated policy rule, including 88 cases accepted by native tests. Nine assertion kills have no new site-associated rule. This illustrates differing observations under the recorded conditions, not scanner superiority or exhaustive complementarity: tfsec does not consume the native test runs and synthetic test inputs, default ignore behavior is retained, downloaded modules are excluded, and line association can miss relevant findings. The scanner's parser failures further limit coverage. The individual repository findings, failed diagnostics and case associations are available in the supplement. For a concrete historical contrast, an edit to `block_public_acls` from true to false at `bootstrap/main.tf:247` in `aws-platform-starter` survives native tests while newly triggering `aws-s3-block-public-acls`. The same field edit at line82 of `terraform/modules/aws-ml-artifact-store/main.tf` in `platform-design` is an assertion kill without a new site-associated rule. These saved catalog2 records illustrate different observations, not proof of deployed vulnerability; exact mutant identities and receipts are in the historical supplement.

6. Discussion and threats to validity

Construct validity. Oasis measures sensitivity to a bounded literal fault model. General Boolean edits, source restrictions and operator overlap shape the denominator; a family name is not proof of a real vulnerability. Runtime rejection differs from an assertion oracle. Selected-context binding removes some no-ops but does not solve semantic equivalence or measure dynamic resource/attribute coverage. No real Terraform fault corpus validates predictive accuracy. Policy findings, survivors and deployed-cloud vulnerabilities are distinct observations.

Internal validity. Fixtures, mock outputs, pre-existing source adaptations, aliases, dependency re-resolution and command timeouts affect outcomes. The historically adapted 23 sources are not pristine upstream tests. Current expansion assertions are unchanged, but a passing mock baseline cannot certify a live deployment. Native early failure can skip later runs. Hash/signature checks support restoration and evidence identity, not freedom from every provider/engine defect. Failed transport/setup attempts are retained; no assertion is weakened to make a baseline pass.

Selection and statistical validity. Public mock-compatible projects, shared providers/modules, vulnerable fixtures, short-workload ordering and the visible-results deadline cutoff limit generalization. Unrun and preparation-failed repositories remain in the selection ledger. Cases repeat edits across scopes and cluster by repository/family;

pooled fractions and repository summaries are descriptive. A narrow binomial interval would not address nonrandom selection. The matched experiment has development exposure, a small non-blind transfer set and investigator-authored treatment; it cannot demonstrate universal assertion improvement.

Comparison and performance validity. Catalog 3 adds sites/operators and changes conservative binding relative to historical runs. The retained 12 view is a within-catalog 3 subset, not a fair historical-version benchmark. Static scanner location matching can miss indirect findings or associate unrelated checks; parser failures and unequal native/scanner input contexts prevent precision/recall claims. Timings are single observations with warm caches, concurrency and excluded overhead. The auxiliary third-lane baseline is not mixed into primary costs. No estimate is presented as repeatable cross-engine performance.

7. Reproducibility and future work

The source repository is `https://github.com/DegenerateUSER/Oasis`. Primary execution freezes tool 0.3.0-research.1/catalog 3 with 22 source hashes supplied in the supplement. Exact upstream/private snapshot revisions, operator/site identities, selected commands, provider locks, rawCSV, diagnostics, native timings, source-restoration checks and archive SHA256/member inventories are retained. The primary archive audit verifies 8186 source files and 30828 members. The original 95 plan and deadline dispositions are supplied. Canonical identity deduplication gives 69 scored identities across historical and primary versions, but this union is not a shared mutation denominator.

Historical execution revision`d535bee1ef9bb1f62c9c258dd67ead715ba826b3` and matched-harness revision`7124220311bf35dcf77b6a3438fdb8cb029b59a2` remain identified as local research revisions, without claiming publication upstream or a newly assigned DOI. The frozen 123-case CSV hash and original verification receipts are retained; the revision's preservation audit is not a rerun. Primary framework validation records 242 passed checks and 4 skipped opt-in provider checks, plus Terraform/OpenTofu native module/default/capacity canaries excluded from research. Credentials and provider binaries are not part of the publication source bundle.

Future work should evaluate engine-specific constructs, broader interfaces, expression/object mutations, dynamic effective coverage, manually assessed survivor causes, independent cohorts and real-fault coupling. Additional scanners and repeated cost measurements require separate controlled experiments. They are not implied by completing this deadline-limited study.

8. Conclusion

The primary frozen-catalog 3 study archives 62 repository attempts; 47 yield 1288 scored cases with 115 assertion kills, 76 runtime-error kills and 1097 survivors. Full selected-baseline coverage and partial coverage remain separate, as do 245 excluded mutant records, blocked scopes and unrun repositories. Generic Boolean mutations dominate the pool, constraining security-specific conclusions. A separate matched 123-case study increases assertion detections from 39 to 73 without lost detections. These results support mutation feedback as a diagnostic aid for selected native IaC oracles, with explicit attention to fixtures, fault models, attrition and cost. They establish neither representative security prevalence nor a universal adequacy threshold.

Declaration of AI assistance

OpenAI Codex and Hermes Agent (Nous Research) were used for implementation assistance, analysis support, manuscript drafting and LaTeX preparation. AI outputs do not independently validate experimental results. The authors are responsible for verifying the research record and final text; final coauthor review and confirmation of this declaration are required before submission.

Acknowledgements

The authors acknowledge the Department of Information Technology, Maharaja Surajmal Institute of Technology, Janakpuri, New Delhi. This work was not supported by research grants.

References

[1] A. Rahman and L. Williams, Source code properties of defective infrastructure as code scripts, Information and Software Technology 112 (2019) 148–163. https://doi.org/10.1016/j.infsof.2019.04.013.

[2] A. Rahman, C. Parnin, and L. Williams, The seven sins: Security smells in infrastructure as code scripts, in: ICSE, 2019, pp. 164–175. https://doi.org/10.1109/ICSE.2019.00033.

[3] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, Assessing the adoption of security policies by developers in terraform across different cloud providers, Empirical Software Engineering 30 (2025), article 74. https://doi.org/10.1007/s10664-024-10610-0.

[4] HashiCorp, Tests, Terraform language documentation. https://developer.hashicorp.com/terraform/language/tests. Accessed 7 November 2026.

[5] OpenTofu, Command: test, OpenTofu documentation. https://opentofu.org/docs/cli/commands/test/. Accessed 7 November 2026.

[6] HashiCorp, Tests: provider mocking, Terraform language documentation. https://developer.hashicorp.com/terraform/language/tests/mocking. Accessed 7 November 2026.

[7] Y. Jia and M. Harman, An analysis and survey of the development of mutation testing, IEEE Transactions on Software Engineering 37 (5) (2011) 649–678. https://doi.org/10.1109/TSE.2010.62.

[8] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, Mutation testing advances: An analysis and survey, Advances in Computers 112 (2019) 275–378. https://doi.org/10.1016/bs.adcom.2018.03.015.

[9] G.-P. Drosos, T. Sotiropoulos, G. Alexopoulos, D. Mitropoulos, and Z. Su, When your infrastructure is a buggy program: Understanding faults in infrastructure as code ecosystems, Proceedings of the ACM on Programming Languages 8 (OOPSLA2) (2024), article 359. https://doi.org/10.1145/3689799.

[10] N. Saavedra and J. F. Ferreira, GLITCH: Automated polyglot security smell detection in infrastructure as code, in: ASE, 2022, pp. 1–12. https://doi.org/10.1145/3551349.3556945.

[11] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira, and A. Mendes, Polyglot code smell detection for infrastructure as code with GLITCH, in: ASE, 2023, pp. 2042–2045. https://doi.org/10.1109/ASE56229.2023.00162.

[12] D. Sokolowski, D. Spielmann, and G. Salvaneschi, Automated infrastructure as code program testing, IEEE Transactions on Software Engineering 50 (6) (2024) 1585–1599. https://doi.org/10.1109/TSE.2024.3393070.

[13] D. Spielmann, G. Zakhour, D. Arnold, M. Biagiola, R. Meier, and G. Salvaneschi, Metamorphic testing for infrastructure-as-code engines, Proceedings of the ACM on Programming Languages 10 (OOPSLA1) (2026), article 118. https://doi.org/10.1145/3798226.

[14] L. Madeyski, W. Orzeszyna, R. Torkar, and M. Józala, Overcoming the equivalent mutant problem: A systematic literature review and a comparative experiment of second order mutation, IEEE Transactions on Software Engineering 40 (1) (2014) 23–42. https://doi.org/10.1109/TSE.2013.44.

[15] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, Are mutants a valid substitute for real faults in software testing?, in: FSE, 2014, pp. 654–665. https://doi.org/10.1145/2635868.2635929.

[16] L. Inozemtseva and R. Holmes, Coverage is not strongly correlated with test suite effectiveness, in: ICSE, 2014, pp. 435–445. https://doi.org/10.1145/2568225.2568271.

[17] G. Petrović, M. Ivanković, G. Fraser, and R. Just, Does mutation testing improve testing practices?, in: ICSE, 2021, pp. 910–921. https://doi.org/10.1109/ICSE43902.2021.00087.

[18] G. Petrović, M. Ivanković, G. Fraser, and R. Just, Practical mutation testing at scale: A view from Google, IEEE Transactions on Software Engineering 48 (10) (2022) 3900–3912. https://doi.org/10.1109/TSE.2021.3107634.

[19] A. B. Sánchez, J. A. Parejo, S. Segura, A. Durán, and M. Papadakis, Mutation testing in practice: Insights from open-source software developers, IEEE Transactions on Software Engineering 50 (5) (2024) 1130–1143. https://doi.org/10.1109/TSE.2024.3377378.

[20] Aqua Security, tfsec, version 1.28.14, source repository and release. https://github.com/aquasecurity/tfsec/releases/tag/v1.28.14. Accessed 7 November 2026.